Introduktion till C++



Uppgift 9 – Input

Uppgiftens målsättning

Den studerande ska få skriva en Inputhanterare som med hjälp av Windows meddelandeloop kan känna av input från tangentbord och mus samt tillhandahålla funktioner för att enkelt arbeta med denna information.
Den studerande ska även få öva på att googla sig till en lösning utifrån ett nytt problem.

Uppgift

Skapa en inputhanterare som kan användas i spel och uppgifter.
Programmet måste reagera på input i realtid.

Din inputhanterare kommer att ha ett interface i stil med:

```
bool IsKeyDown(const int aKeyCode) const;
bool IsKeyPressed(const int aKeyCode) const;
POINT GetMousePosition() const;
etc.
```

Implementationen ska göras som en ny klass i ert Common Utilities bibliotek ni redan påbörjat.

Ett nytt projekt ska skapas för att testa att inputhanteraren fungerar som väntat. I detta testprojekt inkluderar ni ert CommonUtilities-bibliotek för att få tillgång till Inputhanteraren.

För att Inputhanteraren ska vara användbar i andra projekt, vill vi att så lite kod som möjligt ska behöva skrivas för att få igång den i ett projekt. Därför skapar vi följande medlemsfunktion i klassen:

```
bool UpdateEvents(UINT message, WPARAM wParam, LPARAM lParam);
```

Den ska returnera true om meddelandet hanterades och false om meddelandet *inte* hanterades.
När vi då skapar en ny Windowsapplikation behöver vi endast skapa en variabel, ex. inputHandler, samt lägga till två rader kod högst upp i funktionen `WndProc` för att få igång vår inputhanterare:

```
if ( inputHandler.UpdateEvents(messate, wParam, lParam) )
    return 0;
```

I denna funktion `UpdateEvents` som ska ligga i er InputHandlerklass, ska ni läsa av vilket meddelande som ni fick.
Alla meddelande kommer att tas emot, även de som inte är relevanta för er input hanterare (t.ex. att fönstret flyttats).
Då ska funktionen returnera false vilket innebär att meddelandet inte hanterats. Meddelandet kan då hanteras av en annan del av programmet istället.
Om meddelandet är av inputkaraktär (t.ex. en tangent trycktes ner), så ska er inputhandler ta vara på denna information och returnera true istället.

Information om inputen måste sparas i klassen så att man senare i programmet kan fråga klassen t.ex. om tangenten är nertryckt eller ej.

Features

Er Inputhanterare ska **minimum** ha interface för följande:

1. Kolla om en specifik tangent är nedtryckt just nu (användaren tryckte ner den precis nu).

2. Kolla om en specifik tangent är nertryckt hela tiden (användaren håller inne en tangent).

3. Kolla om en specifik tangent precis släpptes.

4. Hämta musens förflyttning sedan föregående uppdatering

5. Kolla om en specifik musknapp är nedtryckt. Det måste finnas stöd för att kolla vänster, höger och mittenknappen

Om ni vill ha er inputhanterare riktigt **funktionsduglig** så är det starkt rekommenderat att ni även implementerar följande:

- Kolla scrollhjulets förflyttning sedan föregående uppdatering

- Kolla om en specifik musknapp precis blev nedtryckt (dvs. sedan föregående uppdatering)

- Kolla om en specifik musknapp precis släpptes upp (dvs. sedan föregående uppdatering)

- Kolla musens absoluta position (använd Windows inbyggda funktioner) i förhållande till windowsfönstret.

- Sätta musens position (använd Windows inbyggda funktioner)

- Capture mouse, dvs. lås musen till programmets windowsfönster. Detta förhindrar att användaren råkat flytta muspekaren utanför fönstret i dina spel och klickar på andra saker (om man har två skärmar eller inte kör i fullscreen).

Testprogram

För att kolla att er Inputhanterare gör vad den ska behöver ni skapa ett nytt program som testar detta och skriver ut resultaten i ett konsolfönster.

I detta projekt kommer vi att testa manuellt då det ger oss tillfälle att titta på Windowsapplikationer och meddelandeloopen.

Att testa automatiskt med enhetstester hade dessutom blivit lite krångligare eftersom vi hade behövt simulera tangenttryckningar.

För att skapa ett projekt för en Windowsapplikation väljer ni Win32 Project som visades under föreläsningen. En Windowsapplikation har inte en konsol så det får vi lägga till för att kunna skriva ut information, vilket även det visades under föreläsningen.
Er Inputhanterare ska länkas in från CommonUtilities-biblioteket som vanligt.

En instans av vår inputhanterare måste skapas.
För denna uppgift är det ok att denna variabel är global i main-filen eftersom det kan vara krångligt att skicka in den till Windowsproceduren.

Efter att fönstret har initierats (if-satsen med InitInstance) är ett bra ställe att anropa Inputhanterarens initialiseringsfunktion, om det finns en sådan. Därefter behöver Inputhanteraren uppdateras i Windowsproceduren (`WndProc`) genom att överst lägga till två rader kod (om vår variabel för Inputhanteraren heter inputHandler):

```
if ( inputHandler.UpdateEvents(message, wParam, lParam) )
     return 0;
```

Koden för att testa Inputhanteraren bör ni lägga i main-funktionen och med fördel i meddelandeloopen:

```
// Main message loop:
while (GetMessage(&msg, nullptr, 0, 0))
{
    if (!TranslateAccelerator(msg.hwnd, hAccelTable, &msg))
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    // Lägg till testkod här

}
```

Testkoden ska anropa de olika funktionerna i Inputhanteraren och skriva ut information om testerna i konsolen. Ni behöver även skriva ut, antingen i konsolen eller i en meddelanderuta, vilka tangenter och musknappar som används för att testa de olika funktionerna.
Varje punkt under *Features* ska ha minst en tangent/musknapp som testar den.

Tips

- Börja med att få grepp om hur windows meddelande-loopen fungerar.
- Ha er InputHandler global så den går lätt att komma åt överallt.
- InitInstance(...) Här kan ni skapa Console för debug output.
- Lägg in följande i wWinMain -> "Main message loop:"

```
globalInputManager.UpdateInput();
if (globalInputManager.GetKeyDown(Keys::MOUSELBUTTON))
{
    std::cout << "left mouse pressed" << std::endl;
    //Gäller bara om ni använder Console output
}
//osv...
```

- Updatera Events:en i WndProc(...) lägg till:

```
if (globalInputHandler.UpdateEvents(message, wParam, lParam))
{
    return 0;
}
```

Mall

För att ytterligande underlätta interface får du här en mall av en header fil som visar hur klassen ser ut.

```
namespace CommonUtilities
{
    class InputHandler
    {
    public:
        InputHandler();

        bool UpdateEvents(UINT message, WPARAM wParam, LPARAM lParam);

        bool IsKeyDown(const int aKeyCode) const;

        bool IsKeyPressed(const int aKeyCode) const;

        bool IsKeyReleased(const int aKeyCode) const;

        bool UpdateMouseInput(UINT aMessage);

        void UpdateInput();

    private:
        std::array<bool, 256> myCurrentState;
        std::array<bool, 256> myPreviousState;
        std::array<bool, 256> myInputState;

    };
}
```

Windows.h innehåller defines för tangentbordet och musen. Exempel:

WM_LBUTTONDOWN = 0x0201 (Vänster musknapp nedtryckt)
VK_LBUTTON = 0x01 (Vänster musknapp)

WM_KEYDOWN = 0x0100 (Tangent nedtryckt)
VK_SPACE = 0x20 (Space)

Läs mer:

- https://learn.microsoft.com/en-us/windows/win32/inputdev/mouse-input
- https://learn.microsoft.com/en-us/windows/win32/inputdev/keyboard-input
- https://learn.microsoft.com/en-us/windows/win32/inputdev/virtual-key-codes

Inlämning

Ni ska som vanligt lämna in en *Exe*-mapp där det finns en körbar version av testprogrammet. I *Source* ska ni lämna in två projekt: *både* testprogrammet och CommonUtilities-projektet.